

Implementing Kurushima's Algorithm in Python

In the previous sections, we discussed how to perform our algorithms of interest “by hand” before providing details of the implementation of the code. In the case of Kurushima’s algorithm, we’ll outline the steps and introduce the code simultaneously. The reason for this change is the relative complexity of the algorithm, and especially the length of the code required to implement it.

One of the most elegant algorithms for generating magic squares, *Kurushima’s algorithm* is named for Kurushima Yoshita, who lived during the Edo period. Kurushima’s algorithm works only for magic squares of *odd dimension*, meaning that it works for any $n \times n$ square if n is an odd number. It begins by filling out the center of the square in a way that matches the Luo Shu square. In particular, the central five squares are given by the following expressions, with n here referring to the dimension of the square ([Table 2-10](#)).

Table 2-10: The Center of Kurushima's Square

	n^2	
n	$(n^2 + 1)/2$	$n^2 + 1 - n$
1		

Kurushima’s algorithm for generating an $n \times n$ magic square for odd n can be described simply as follows:

1. Fill in the five central squares according to [Table 2-10](#).
2. Beginning with any entry whose value is known, determine the value of an unknown neighboring entry by following one of the three rules (described next).
3. Repeat step 2 until every entry in the full magic square is filled in.

Filling in the Central Squares

We can begin the process of creating a magic square by creating an empty square matrix that we’ll fill up. For example, if we want to create a 7×7